SaddlepointExpansions for MATLAB

User manual

Simon A. Broda and Raymond Kan

October 2026

Contents

1 Introduction **2**

2 A first example **2**

3 Conventions **3**

4 Cumulant generating functions **3**
4.1 The class CGF 3
4.2 The library 4
4.3 Transformations 4
4.4 Writing your own 5
4.5 Closed-form Taylor polynomials 5

5 Orthant probabilities **5**

6 Densities **6**

7 Ratios **6**
7.1 Usage 6
7.2 Examples 7
7.3 Cost 7

8 Derivatives, saddlepoints and the Taylor class **8**

9 The expansions **8**
9.1 Whitened coefficients 8
9.2 Densities 9
9.3 Orthant probabilities 9
9.4 Ratios 9

10 Accuracy and precision **10**

11 The engine **10**

12 Validation **10**

13 Function reference **11**

1 Introduction

SaddlepointExpansions for MATLAB computes saddlepoint expansions of any order for the distribution of a random vector from its cumulant generating function alone. It implements the expansions of Broda and Kan, "Uniform saddlepoint expansions for multivariate tail probabilities":

- the orthant probability $\mathrm{P}(X \geq x)$ of a random vector X of any dimension d, with lower and mixed corners, and with the inclusion–exclusion step for saddlepoints that have negative components;
- the density of X in any dimension;
- the joint density and the joint distribution function of a vector of ratios $R = (X_1/Y_1, \ldots, X_d/Y_d)$ with positive denominators, including ratios with a common denominator;
- in one dimension, the expansions of Daniels (density), Hauschildt and Robinson (distribution function) and Rice (density of a ratio).

The approximation of order M has relative error $O(n^{-M})$, where n is the large parameter (a sample size, a number of degrees of freedom).

The user writes the cumulant generating function K as an ordinary MATLAB function. Its derivatives of all orders are generated by Taylor-mode automatic differentiation (the class `tps`, truncated multivariate power series with overloaded arithmetic and elementary functions), directly in the whitened coordinates in which the expansions are evaluated, so that no derivative has to be written by hand and none is computed by finite differences. The expansions themselves are computed by the MATLAB programs that accompany the paper (folder `engine`), which can also be called directly with closed-form derivatives.

The package is the MATLAB counterpart of the Julia package `SaddlepointExpansions.jl` by Simon Broda (`https://github.com/s-broda/SaddlepointExpansions.jl`), which also has a Python version (`https://github.com/s-broda/saddlepoint-expansions`). The packages have the same interface and share a set of reference values, on which they agree to about 10^{-13} relative (§ 12); the Julia package also runs in variable precision (§ 10).

Theorem and corollary numbers refer to the version of the paper of October 2026: Theorem 1 (orthant probability), Theorem 2 (inclusion–exclusion), Theorem 3 (density), Theorem 4 and Corollary 1 (densities of ratios), Corollaries 2 to 5 (Daniels, Hauschildt–Robinson, the distribution function of a ratio, Rice).

Requirements. MATLAB (the package was tested with R2025b) and the Statistics and Machine Learning Toolbox, whose `normcdf` and `normpdf` are used by the univariate engine. Nothing else is needed.

Installation. Copy the folder anywhere and run `spx_setup`, which adds the folder and its `engine` subfolder to the path. `help SaddlepointExpansions` lists the functions, `examples/demo_spx.m` runs the examples of this manual in a few seconds and `tests/test_spx.m` the tests of § 12 in about a minute.

2 A first example

Let $X = (U_0 + U_1, U_0 + U_2)$ with U_0, U_1, U_2 independent unit exponential variables, the example of Kolassa and Li (2010), and let $\bar{X}$ be the mean of $n = 5$ independent copies of X. Then

```
spx_setup
K = CGF(@(t) -log(1 - t(1)) - log(1 - t(2)) - log(1 - t(1) - t(2)), 2, ...
        'domain', @(t) t(1) < 1 && t(2) < 1 && t(1) + t(2) < 1);
Kn = sample_mean(K, 5);               % the cgf 5 K(t/5) of the mean
r = tailprob(Kn, [3 3], 'order', 4);  % P(mean1 >= 3, mean2 >= 3)
r.F'                                  % 0.02142343 0.02214365 0.02215413 0.02215193
r.tau                                 % the saddlepoint
s = density(Kn, [3 3], 'order', 4);   % the joint density of the mean at (3, 3)
```

The exact probability is 0.02215196; the approximations of orders one to four have relative errors -3.3×10^{-2}, -3.8×10^{-4}, 9.8×10^{-5} and about -1.4×10^{-6}. The same cumulant generating function is in the library as `exponential_sums_cgf([1 0; 1 1; 0 1], 'n', 5)`.

3 Conventions

The large parameter. The large parameter n is absorbed in the cumulant generating function: the expansions are those of the vector whose cumulant generating function is supplied. For the mean of n copies one supplies $nK(t/n)$, which `sample_mean(K, n)` constructs; the density returned is then that of the mean.

Orders. Every expansion function takes `'order', M` (default three) and returns the approximations of orders $1, \dots, M$ in a column vector. The approximation of order M has relative error $O(n^{-M})$. For a density it keeps the terms $m = 0, \dots, M-1$ of the series in powers of n^{-1}, and for an orthant probability the terms $m = 0, \dots, 2M-1$ of the series in powers of $n^{-1/2}$. For densities, order one is Daniels's first-order approximation; for orthant probabilities, the leading term $m = 0$ is the approximation of Kolassa (2003).

Results. The expansion functions return a structure with the approximations (`F` or `f`), the saddlepoint (`tau`, or `shat` for ratios), a `status` string (`'ok'` or a warning about the numerical evaluation) and `info`, the diagnostics of the underlying engine program. The diagnostics measure the numerical quality of the evaluation, such as cancellation among the terms or the change with half the quadrature nodes; they do not measure the asymptotic error of the expansion, which is what the order controls.

Vectors. Points may be row or column vectors. The argument `t` of a cumulant generating function is always a $d \times 1$ column vector.

4 Cumulant generating functions

4.1 The class CGF

```
K = CGF(fun, d)
K = CGF(fun, d, 'domain', pred)
K = CGF(fun, d, 'Taylor', tfun)
```

`fun` is a function of a $d \times 1$ vector `t` returning $K(t)$, also for $d = 1$. `pred` is an optional predicate `@(t) logical` describing the domain, and `tfun` an optional function that returns Taylor polynomials in closed form (§ 4.5).

The function is evaluated on vectors of truncated power series to generate its derivatives, so it must be written with operations that the class `tps` supports:

- `+`, `-`, `.*`, `./`, `.^`, `*` and `/` by numbers, products `A*t` and `t.'*A` with constant matrices, `sum`, `prod`, `dot`, indexing `t(j)`, `t(1:k)`, concatenation, `reshape`, `diag`;
- `exp`, `expm1`, `log`, `log1p`, `sqrt`, `sin`, `cos`, `tan`, `sinh`, `cosh`, `tanh`, `atan`, `abs`, and `sinhc`, `sincu` of the package;
- comparisons (`<`, `<=`, ...), which compare the constant terms, so that code with tests on the argument can be differentiated.

Functions of `t` that call compiled numerical routines (`det`, `inv`, `chol`, `eig`, `expm`, ...) cannot be differentiated this way; §4.4 shows how to write determinants and inverses.

The domain. Outside the domain, `fun` may raise an error with identifier `'tps:domain'` (the logarithm and the square root of a nonpositive number do so automatically) or return a nonfinite value; both are treated as "outside the domain", and the saddlepoint iteration steps back. When the formula is finite outside the true domain, as $1/(1-t)$ is for $t > 1$, the predicate `'domain'` is needed. It is also cheaper than an error.

4.2 The library

constructor	distribution
`normal_cgf(mu, Sigma)`	$N(\mu, \Sigma)$ (the expansions are exact at order one)
`gamma_cgf(a, b)`	gamma with shape a and scale b (default one); `gamma_cgf(1, b)` is the exponential distribution with mean b
`ncx2_cgf(m, lam)`	noncentral chi-squared $\chi^2_m(\lambda)$
`exponential_sums_cgf(A, 'n', n, 'shift', l)`	the mean of n independent copies of $A^\mathsf{T}U + l$, U a vector of independent unit exponential variables
`quadratic_forms_cgf(Mats, 'Sigma', S, 'mu', mu, 'scale', c)`	the quadratic forms $z^\mathsf{T}A_i z$, $i = 1, \ldots, d$, in $z \sim N(\mu, \Sigma)$, `Mats = {A_1,...,A_d}`, summed over c independent copies
`wishart_diag_cgf(V, n)`	the diagonal of a Wishart matrix $W_d(n, V)$
`common_component_cgf(m, lam)`	$Q_j = X_j + X_0$, $j = 1, \ldots, d$, with independent $X_i \sim \chi^2_{m_i}(\lambda_i)$, `m = [m_1 ... m_d m_0]`
`uniform_cgf(a, b)`	uniform on $[a, b]$ (default $[-1, 1]$)

The cumulant generating function of quadratic forms is

$$K(t) = -\frac{1}{2}\log\det(I - 2C) + \nu^\mathsf{T} C (I - 2C)^{-1}\nu, \qquad C = \sum_{i=1}^{d} t_i \Sigma^{1/2} A_i \Sigma^{1/2}, \qquad \nu = \Sigma^{-1/2}\mu,$$

on the set where $I - 2C$ is positive definite. Its Taylor polynomials are computed in closed form by the trace recursion of the paper, which is much faster than automatic differentiation for matrices of large order.

4.3 Transformations

function	cumulant generating function of
`reflect(K, s)`	$(s_1 X_1, \ldots, s_d X_d)$, $s_j = \pm 1$; `reflect(K)` is that of $-X$
`marginal(K, idx)`	the components `idx` of X

function	cumulant generating function of
`linear_combination(K, B)`	$B^{\mathsf{T}}X$ for a $d \times k$ matrix B, that is $K(Bu)$
`sample_mean(K, n)`	the mean of n independent copies, $nK(t/n)$
`shift(K, c)`	$X + c$
`K1 + K2`	the sum of two independent vectors of the same dimension

The transformations keep closed-form Taylor polynomials: a linear combination of quadratic forms, for example, is still differentiated by the trace recursion.

4.4 Writing your own

Removable singularities are the one trap: $\sinh(t)/t$ is $0/0$ at $t = 0$, where the saddlepoint iteration starts. Use `sinhc` and `sincu`, which switch to the power series there; the library defines the uniform distribution as `log(sinhc(t))`. Two patterns:

```
% independent parts, one per component, plus a common part
K = CGF(@(t) -sum(m(:)/2.*log(1 - 2*t)) - m0/2*log(1 - 2*sum(t)), d, ...
        'domain', @(t) all(t < 1/2) && sum(t) < 1/2);

% a log-determinant: a Cholesky factorization written in plain arithmetic
function v = logdet_cgf(t, B1, B2)
n = size(B1, 1); Q = eye(n) - 2*(t(1)*B1 + t(2)*B2);
L = Q(1, 1)*zeros(n);                % a series array of the right class
for j = 1:n
   s = Q(j, j); for k = 1:j-1, s = s - L(j, k)^2; end
   L(j, j) = sqrt(s);
   for i = j+1:n
      w = Q(i, j); for k = 1:j-1, w = w - L(i, k)*L(j, k); end
      L(i, j) = w/L(j, j);
   end
end
v = -sum(log(diag(L)));
end
```

The test file `tests/test_spx.m` differentiates the complete cumulant generating function of a pair of quadratic forms written this way and checks it against the trace recursion. For quadratic forms in normal variables, `quadratic_forms_cgf` is faster.

4.5 Closed-form Taylor polynomials

The option `'Taylor', tfun` supplies a function `tfun(t, N, M)` that returns the Taylor polynomial of degree N in v of $K(t+Mv)$ as a scalar `tps` in `size(M, 2)` variables (`M = []` stands for the identity). It is then used instead of automatic differentiation; `fun` is still used for the values of K. The polynomial is built from its coefficients with `tps(C, [1 1], k, N)`, where `C` holds the coefficients of the monomials in the order of `tps.tables(k, N).mi` (§ 8). `quadratic_forms_cgf` is an example.

5 Orthant probabilities

```
r = tailprob(K, x, 'order', M)                  % P(X >= x)
r = tailprob(K, x, 'order', M, 'lower', true)   % P(X <= x)
r = tailprob(K, x, 'order', M, 'signs', s)      % e.g. s = [1 -1]: P(X1 >= x1, X2 <=
    x2)
```

`r.F` holds the approximations of orders $1, \ldots, M$ and `r.tau` the saddlepoint, $K'(\hat{\tau}) = x$. Lower and mixed corners are computed as upper orthant probabilities of the reflected vector.

Saddlepoints with negative components. The expansion of Theorem 1 applies to saddlepoints in the closed positive orthant $\hat{\tau} \geq 0$. When some components are negative, `tailprob` writes the probability by the inclusion–exclusion identity of Theorem 2 as a signed sum of orthant probabilities of marginals with some components negated, each with a saddlepoint in its closed positive orthant, recursively, and expands every term (§ 9). This keeps the relative error $O(n^{-M})$; a direct evaluation of the formula at such a saddlepoint would have the wrong exponential order. The saddlepoint reported is that of the original problem.

Dimensions. In one dimension the expansion is that of Hauschildt and Robinson (Corollary 3), computed directly for the upper tail. In two dimensions the integrals of the expansion are computed by Gauss–Legendre quadrature (`spa_orthant2`, `spa_bi`), in three by Gauss quadrature on a conditioned grid (`spa_tri`), and in four or more by randomized quasi-Monte Carlo after Genz's sequential conditioning (`spa_mv`); `r.info.err` is then an error estimate (in four or more dimensions a standard error). A trivariate probability takes about a second at order three and tens of seconds at order four or five, mostly in the inclusion–exclusion terms.

Options. Name-value pairs other than `'order'`, `'lower'` and `'signs'` are passed to the engine: `'nG'` (Gauss–Legendre nodes per coordinate), `'qtol'` (quadrature tolerance), `'nGmax'`, `'method'` (`'quad'` or `'qmc'`), `'nqmc'`, `'nshift'`, `'seed'` and, in two dimensions, `'logscale'`, `c`, which returns the probability scaled by e^{-c} for probabilities below the range of double precision. See the help of `spa_bi`, `spa_tri` and `spa_mv`.

6 Densities

```
r = density(K, x, 'order', M)
```

`r.f` holds the approximations of orders $1, \ldots, M$ of the density of X at x (Theorem 3; Daniels's expansion, Corollary 2, in one dimension). The expansion is a finite sum of Hermite polynomials at zero and needs no integration in any dimension. In three or more dimensions the terms of high orders can cancel by many orders of magnitude; `r.info.kappa_pdf` reports the cancellation (§ 10).

7 Ratios

7.1 Usage

Let $R = (X_1/Y_1, \ldots, X_d/Y_d)$ with $Y_j > 0$. The cumulant generating function `K` is that of the vector $(X_1, \ldots, X_d, Y_1, \ldots, Y_d)$, of dimension $2d$, numerators first. For ratios with a common denominator Y it may instead be that of $(X_1, \ldots, X_d, Y)$, of dimension $d + 1$. The number of ratios is the length of the point `r0`.

```
r = ratio_density(K, r0, 'order', M)                  % density of R at r0
r = ratio_tailprob(K, r0, 'order', M)                 % P(R >= r0)
r = ratio_tailprob(K, r0, 'order', M, 'lower', true)  % P(R <= r0)
r = ratio_tailprob(K, r0, 'order', M, 'signs', s)     % a mixed corner
```

`r.f` or `r.F` holds the approximations of orders $1, \ldots, M$, and `r.shat` the saddlepoint of K_W (§ 9.4). For one ratio the density is Rice's expansion (Corollary 5), and for several the expansion of Theorem 4 and Corollary 1, whose term of order one is the approximation of Butler and

Paolella (1998); the distribution function is the orthant probability of § 5 for $W = X - r_0 \circ Y$ (Corollary 4 for one ratio), with all its options.

7.2 Examples

Sample autocorrelations. With $z \sim N(0, I_N)$, $M = I_N - 1_N 1_N^{\mathsf{T}}/N$ and $(A_j)_{ts} = 1/2$ for $|t - s| = j$, the sample autocorrelations at lags one and two are $R_j = z^{\mathsf{T}} M A_j M z / z^{\mathsf{T}} M z$, ratios with the common denominator $z^{\mathsf{T}} M z$. For $N = 50$ at $r_0 = (-1, -1)/\sqrt{50}$:

```
N = 50; M = eye(N) - ones(N)/N;
A = @(j) (diag(ones(N-j, 1), j) + diag(ones(N-j, 1), -j))/2;
K = quadratic_forms_cgf({M*A(1)*M, M*A(2)*M, M});      % dimension d + 1 = 3
r0 = [-1 -1]/sqrt(N);
f = ratio_density(K, r0, 'order', 5).f
F = ratio_tailprob(K, r0, 'order', 5, 'lower', true).F
```

The relative errors of orders one to five, against the exact values of the paper, are

order	1	2	3	4	5
density	-1.4×10^{-3}	7.3×10^{-5}	-1.0×10^{-5}	2.2×10^{-6}	-6.4×10^{-7}
distribution function	-3.7×10^{-2}	1.5×10^{-4}	1.4×10^{-5}	5.8×10^{-7}	-1.6×10^{-7}

These are the values of the corresponding row of the table of sample autocorrelations in the paper, which were computed by a dedicated program.

Gamma ratios with a common denominator. If $X_1, \ldots, X_d, Y$ are independent with $X_j \sim \text{Gamma}(a_j)$ and $Y \sim \text{Gamma}(b)$, the ratios $R_j = X_j/Y$ have the inverted Dirichlet density

$$f_R(r) = \frac{\Gamma(a_1 + \cdots + a_d + b)}{\Gamma(b) \prod_{j=1}^d \Gamma(a_j)} \frac{\prod_{j=1}^d r_j^{a_j - 1}}{(1 + r_1 + \cdots + r_d)^{a_1 + \cdots + a_d + b}}.$$

```
a = [3 4 5]; b = 6;
K = CGF(@(t) -sum(a(:).*log(1 - t(1:3))) - b*log(1 - t(4)), 4, 'domain', @(t) all(t <
    1));
f = ratio_density(K, [0.6 0.5 0.9], 'order', 5).f
```

The relative errors of orders one to five are 7.7×10^{-2}, -3.1×10^{-3}, -1.0×10^{-4}, 1.7×10^{-5} and 3.6×10^{-6}. Multiplying the shapes by n shows the rate $O(n^{-M})$; for $d = 2$ at $r_0 = (0.6, 0.5)$:

n	order 1	order 2	order 3	order 4
1	5.8×10^{-2}	-1.8×10^{-3}	-1.2×10^{-4}	1.3×10^{-5}
2	2.8×10^{-2}	-4.2×10^{-4}	-1.6×10^{-5}	6.6×10^{-7}
4	1.4×10^{-2}	-1.0×10^{-4}	-2.0×10^{-6}	3.7×10^{-8}
8	7.0×10^{-3}	-2.5×10^{-5}	-2.5×10^{-7}	2.2×10^{-9}

Each doubling of n divides the error of order M by about 2^M.

7.3 Cost

The density of several ratios needs one Taylor polynomial of $\mathcal{K}(s, -r \circ s + \tau)$ in the $2d$ variables (u, τ) of § 9.4, of degree $\max(2M, 2M - 2 + d)$. For two or three ratios every order up to five takes one or two seconds, also for quadratic forms in $N = 50$ variables; for four ratios order four takes several seconds. The distribution function costs what an orthant probability of dimension d costs (§ 5).

8 Derivatives, saddlepoints and the Taylor class

```
D = derivatives(K, t, N)       % all derivatives of orders <= N at t
[tau, info] = saddlepoint(K, x)
p = taylor(K, t, N, M)         % Taylor polynomial of K(t + M v) in v, degree N
```

`D.nu` holds the multi-indices ν, one per row in graded order, and `D.value` the derivatives $K^{(\nu)}(t)$, so that `D.value(1)` is $K(t)$. The saddlepoint is computed by Newton's method with backtracking on the convex function $K(\tau) - \tau^{\mathsf{T}} x$ (`spa_saddle`).

The class `tps` represents arrays of polynomials in k variables truncated at total degree N:

```
v = tps.vars(k, N)       % k x 1 array of the variables
p = exp(v(1)) * log(1 + v(2))
coef(p, [1 2])           % nu! times the coefficient of v^nu, the derivative at 0
double(p)                % the constant term
T = tps.tables(k, N);    % T.mi: the multi-indices in graded order
```

A function f of a series $p = p_0 + q$, q without constant term, is evaluated by composition, $f(p) = \sum_{j=0}^{N} f^{(j)}(p_0) q^j / j!$; where f is not analytic at p_0 an error with identifier `'tps:domain'` is raised. The coefficients are stored in `p.C`, one column per element of the array.

9 The expansions

This section states what the package computes. The paper gives the derivations, the conditions and the error bounds; here n is absorbed in K.

9.1 Whitened coefficients

Let $\hat{\tau}$ be the saddlepoint, $K'(\hat{\tau}) = x$, and $K''(\hat{\tau}) = LL^{\mathsf{T}}$ the Cholesky factorization. All terms are computed for the whitened vector $Y = L^{-1}X$, whose cumulant generating function $K_Y(u) = K(L^{-\mathsf{T}}u)$ has its saddlepoint at $\hat{u} = L^{\mathsf{T}}\hat{\tau}$ and Hessian I there. Let $\theta_\nu = K_Y^{(\nu)}(\hat{u})/\nu!$, $|\nu| \geq 3$, be its Taylor coefficients and define $c_{r,\nu}$ by

$$c_{1,\nu} = \theta_\nu, \qquad c_{r,\nu} = \frac{1}{r} \sum_{k=3}^{|\nu|-3(r-1)} \sum_{|\lambda|=k,\ \lambda \leq \nu} \theta_\lambda \, c_{r-1,\nu-\lambda}, \qquad r \geq 2,$$

so that $c_{r,\nu}$ is the coefficient of u^ν in $\Theta(u)^r / r!$, where

$$\Theta(u) = \sum_{k=3}^{\infty} \sum_{|\nu|=k} \theta_\nu u^\nu.$$

The multivariate Hermite polynomials at zero factor into univariate ones,

$$\mathrm{He}_\nu(0) = \prod_{j=1}^{d} \mathrm{He}_{\nu_j}(0), \qquad \mathrm{He}_{2i}(0) = (-1)^i \prod_{j=1}^{i} (2j-1), \qquad \mathrm{He}_{2i+1}(0) = 0.$$

The package obtains the θ_ν by differentiating the composed function $K(\hat{\tau} + L^{-\mathsf{T}}u)$ directly. In the original coordinates the terms cancel much more when the components of X are strongly correlated under the tilted distribution, and the substitution of $L^{-\mathsf{T}}$ into the Taylor polynomial of K can lose many digits at high degree.

9.2 Densities

$$f(x) \approx \frac{\exp\{K(\hat\tau) - \hat\tau^{\mathsf T} x\}}{(2\pi)^{d/2}|K''(\hat\tau)|^{1/2}} \sum_{m=0}^{M-1} a_m, \qquad a_0 = 1, \qquad a_m = \sum_{r=1}^{2m} \sum_{|\nu|=2m+2r} c_{r,\nu} \mathrm{He}_\nu(0).$$

The coefficient a_m involves the θ_ν with $|\nu| \le 2m+2$.

9.3 Orthant probabilities

For $\hat\tau \ge 0$,

$$\mathrm{P}(X \ge x) \approx \exp\{K(\hat\tau) - \hat\tau^{\mathsf T} x + |\hat u|^2/2\} \sum_{m=0}^{2M-1} T_m, \qquad T_0 = \tilde I_0, \qquad T_m = \sum_{r=1}^{m} \sum_{|\nu|=m+2r} c_{r,\nu} \tilde I_\nu,$$

with

$$\tilde I_\nu = \int_{\{v \in \mathbb{R}^d:\ Lv \ge 0\}} \prod_{j=1}^{d} \mathrm{He}_{\nu_j}(v_j)\, \phi_d(v + \hat u)\, dv,$$

ϕ_d the standard normal density in d dimensions. The term T_m is of order $n^{-m/2}$. When $\hat\tau$ has negative components, let $\mathcal{N} = \{j : \hat\tau_j < 0\}$. Then

$$\mathrm{P}(X \ge x) = \sum_{T \subseteq \mathcal{N}} (-1)^{|T|}\, \mathrm{P}\big(\tilde X_{\mathcal{N}^c \cup T} \ge \tilde x_{\mathcal{N}^c \cup T}\big),$$

where $\tilde X$ and $\tilde x$ are X and x with the components in T negated and the subscript selects components; the term with $\mathcal{N}^c \cup T$ empty is one. Each term has a saddlepoint in its closed positive orthant, or is decomposed again.

9.4 Ratios

Let $\mathcal{K}(s,t)$, $s,t \in \mathbb{R}^d$, be the joint cumulant generating function of (X,Y), $W_j = X_j - r_j Y_j$, and $r \circ s = (r_1 s_1, \ldots, r_d s_d)$. The cumulant generating function of W is $K_W(s) = \mathcal{K}(s, -r \circ s)$, and $\mathrm{P}(R \le r) = \mathrm{P}(W \le 0)$ is the orthant probability of $-W$ at zero. For the density, let

$$\mathcal{K}_B = \frac{\partial^{|B|}\mathcal{K}}{\prod_{j \in B} \partial t_j}, \qquad B \subseteq \{1, \ldots, d\},$$

and let $\mathcal{P}_{d,j}$ be the set of partitions of $\{1, \ldots, d\}$ into j blocks and

$$g_k(s) = \sum_{\pi \in \mathcal{P}_{d,d-k}} \prod_{B \in \pi} \mathcal{K}_B(s, -r \circ s), \qquad k = 0, \ldots, d-1.$$

With $\hat s$ the saddlepoint of K_W, $K_W'(\hat s) = 0$, and $K_W''(\hat s) = LL^{\mathsf T}$,

$$f_R(r) \approx \frac{\exp\{K_W(\hat s)\}}{(2\pi)^{d/2}|K_W''(\hat s)|^{1/2}} \sum_{m=0}^{M-1} \tilde a_m, \qquad \tilde a_m = \sum_{k=0}^{\min(m,d-1)} b_{m-k}(g_k),$$

$$b_m(h) = \sum_{r=0}^{2m} \sum_{j=3r}^{2m+2r} \sum_{|\nu|=j} \sum_{|\mu|=2m+2r-j} c_{r,\nu}\, \gamma_\mu\, \mathrm{He}_{\nu+\mu}(0), \qquad c_{0,0} = 1,$$

where θ_ν, and hence $c_{r,\nu}$, are the whitened coefficients of K_W at $\hat s$ and γ_μ the Taylor coefficients of $h(\hat s + L^{-\mathsf T} u)$ in u. The leading coefficient is $\tilde a_0 = g_0(\hat s) = \prod_{j=1}^{d} \mathcal{K}_{\{j\}}(\hat s, -r \circ \hat s)$. All the coefficients come from one Taylor polynomial of

$$\mathcal{K}\big(\hat s + L^{-\mathsf T} u,\ -r \circ (\hat s + L^{-\mathsf T} u) + \tau\big)$$

in (u, τ): the coefficient of $\prod_{j \in B} \tau_j$ is the Taylor polynomial of $\mathcal{K}_B$ in u. The sums b_m are evaluated as He-weighted sums of the coefficients of the products $\Theta(u)^r\, h(u)/r!$.

10 Accuracy and precision

The computations are in double precision. Rounding errors are about 10^{-14} relative in the tests, far below the asymptotic errors of the expansions at the orders used in practice (10^{-3} to 10^{-9}), so double precision suffices for nearly all purposes. It matters in three situations:

- *Cancellation in three or more dimensions.* At high orders the terms can cancel by many orders of magnitude. The engine reports this (`kappa` in `info`) and, when the cancellation exceeds 10^6, tries to repeat the computation in variable precision. The Taylor class works in double precision only, so in this package that refinement is not possible: the double-precision values are kept and `status` reports `'refinement failed'`. The Julia package repeats the computation in `BigFloat`.
- *Probabilities below about* 10^{-308}. In two dimensions use `'logscale'` (§ 5).
- *Tables of very small relative errors*, at large n and high order, need exact values and expansions beyond 15 digits. The engine programs accept derivatives in variable precision (`vpa`, Symbolic Math Toolbox) when they are supplied in closed form (§ 11).

11 The engine

The folder `engine` contains the programs of the paper. They take the derivatives of K from a function `Kfun(nu, t)`, the derivative of order `nu` ($1 \times d$) at `t` ($1 \times d$), `NaN` outside the domain, and can be used directly when closed-form derivatives are available:

`spa_saddle`	saddlepoint by Newton's method with backtracking
`spa_uni`	Daniels and Hauschildt–Robinson expansions from $K^{(k)}(\hat{\tau})$
`spa_ratio_pdf`	Rice's expansion of the density of a ratio
`spa_bi`, `spa_orthant2`	bivariate density and orthant probability
`spa_tri`, `spa_mv`, `spa_orthant`	three and more dimensions
`qf_Kmulti`	derivatives of the cgf of quadratic forms
`ncx2_spa`, `ncf_spa`, `dncf_spa`	noncentral χ^2, F and doubly noncentral F
`spa_uni_hp`, `spa_bi_hp`	variable precision

The class `CGF` builds `Kfun` and the whitened derivatives (`opts.Kwhite`) from the Taylor polynomials. See the help of each program.

12 Validation

`tests/test_spx.m` runs the following checks and returns `true` if all pass (`test_spx(true)` prints every result):

- derivatives of the Taylor class against closed forms and finite differences, and the derivatives of a pair of quadratic forms by automatic differentiation through a generic Cholesky factorization against the trace recursion;
- the 409 reference values shared by the Julia and Python packages (`tests/julia_reference.csv`, generated by `test/reference_values.jl` of the Julia package; `tests/julia_reference.jl` copies them): derivatives, saddlepoints, orthant probabilities with lower and mixed corners and saddlepoints with negative components, and densities in one to four dimensions (sums of exponentials including the examples of Wang and of Kolassa and Li, gamma, noncentral chi-squared, uniform, common-component, quadratic-form, Wishart and normal distributions), single ratios

(the noncentral F) and several ratios. They agree to about 10^{-13} relative, at most 10^{-11}; the four-dimensional orthant probabilities, computed by quasi-Monte Carlo, agree within three standard errors;

- the expansion for several ratios at $d = 1$ against Rice's expansion (10^{-14}), and for two and three ratios against the inverted Dirichlet density and distribution function and the product of two independent beta-prime ratios, including the rate $O(n^{-4})$ of the density of order four.

In addition, for all 24 points of the table of sample autocorrelations of the paper, the density and distribution function of two ratios computed by the package agree with those of the dedicated program of the paper (`acf_spa`) to 1.5×10^{-12} at every order from one to five.

13 Function reference

call	result
`K = CGF(fun, d, 'domain', pred, 'Taylor', tfun)`	a cumulant generating function
`r = tailprob(K, x, 'order', M, 'lower', tf, 'signs', s, ...)`	orthant probabilities; `r.F`, `r.tau`, `r.status`, `r.info`
`r = density(K, x, 'order', M, ...)`	densities; `r.f`, `r.tau`, `r.status`, `r.info`
`r = ratio_density(K, r0, 'order', M)`	densities of $d =$ `numel(r0)` ratios; `r.f`, `r.shat`, `r.info` (for $d \geq 2$: the terms `a`, the log prefactor `logC`)
`r = ratio_tailprob(K, r0, 'order', M, 'lower', tf, 'signs', s, ...)`	distribution functions of ratios; `r.F`, `r.shat`, `r.status`, `r.info`
`[tau, info] = saddlepoint(K, x)`	the saddlepoint
`D = derivatives(K, t, N)`	`D.nu`, `D.value`
`p = taylor(K, t, N, M)`	Taylor polynomial of $K(t + Mv)$
`v = eval(K, t)`	$K(t)$
`reflect`, `marginal`, `linear_combination`, `sample_mean`, `shift`, `+`	transformations (§ 4)
`tps.vars(k, N)`, `tps.const(c, k, N)`, `coef(p, nu)`, `double(p)`, `tps.tables(k, N)`	the Taylor class
`sinhc(x)`, `sincu(x)`	$\sinh(x)/x$ and $\sin(x)/x$
`spx_setup`	adds the package to the path

References

Broda, S. A. and Kan, R. Uniform saddlepoint expansions for multivariate tail probabilities. Working paper, 2026.

Butler, R. W. (2007). *Saddlepoint Approximations with Applications.* Cambridge University Press.

Butler, R. W. and Paolella, M. S. (1998). Approximate distributions for the various serial correlograms. *Bernoulli* 4, 497–518.

Daniels, H. E. (1954). Saddlepoint approximations in statistics. *Ann. Math. Statist.* 25, 631–650.

Hauschildt, R. (1969). Saddlepoint approximations to distribution functions. Master's thesis, University of British Columbia, Vancouver.

Kolassa, J. E. (2003). Multivariate saddlepoint tail probability approximations. *Ann. Statist.* 31, 274–286.

Kolassa, J. E. and Li, J. (2010). Multivariate saddlepoint approximations in tail probability and conditional inference. *Bernoulli* 16, 1191–1207.